Dropbox to GCS Migration: Impact & Developer Effort

Blast-radius analysis and phased implementation plan for Ad Genius

This document has three parts. **Part 1** maps the impact (blast radius) of moving Ad Genius's video assets from Dropbox to Google Cloud Storage (GCS): every area of the system that is affected, what changes, what could break, and how risky it is. **Part 2** breaks the implementation into phases with a developer-focused effort estimate. **Part 3** lists the team's actual requirements and how each one would be rebuilt.

The message to take away up front: this is **not a simple storage swap — it is a large, complexity-adding programme.** It spans two codebases (the Next.js app and the Python backend), a high-risk data migration, and rebuilding features from **three separate products** — Dropbox, Dropbox Replay, and Air — two of which offer no API to integrate against.

> Reading guide: in Part 1 each affected area shows **Today**, **After GCS**, **Could break**, and **To update**, with a **Risk** badge and a one-line **Recommendation**. Effort in Parts 2 and 3 uses T-shirt sizes (Small / Medium / Large / Very Large) reflecting build complexity and scope. AI-assisted development compresses the coding, but not the data migration, the testing, or the effort of rebuilding three products' worth of features.

Part 1 — Migration impact / blast radius

1.1 Current Dropbox-based upload & sync flow · Risk: Medium

Today	Editors upload finished videos to a shared Dropbox folder, paste the shared link on the video record, then run a manual “Fetch Dropbox Folder” action that scans the folder and matches files to records by name.
After GCS	Editors upload directly inside Ad Genius; the browser sends the file straight to a private GCS bucket via a signed upload, and the record is created immediately with a GCS key — no paste, no separate scan/match step.
Could break	Anything assuming a video always has a Dropbox link, or that depends on the folder-sync to populate links.
To update	The video-create UI, the folder-sync route (retire/rename), and every place that reads `dropboxLink` as the media source.

Recommendation: Keep both flows during the transition; default new uploads to GCS and leave the sync working for legacy videos.

1.2 Video records & database schema · Risk: High

Today	`Video.dropboxLink` is required; several models (iteration suggestions, winning-hooks, winning-concept analysis) store the normalized Dropbox URL, some with unique constraints and indexes on it.
After GCS	`Video.gcsKey` (already added) holds the object; a source-provider field distinguishes Dropbox vs GCS; a Dropbox-free identifier becomes the canonical key; `dropboxLink` becomes optional/legacy.
Could break	Unique constraints and de-dupe keyed on the Dropbox URL if switched carelessly; the NOT-NULL on `dropboxLink` for new GCS videos.
To update	Prisma + SQLAlchemy schema: make `dropboxLink` optional, add `sourceProvider`/`sourceUri`, and migrate the identity columns in a controlled step.

Recommendation: Do additive changes first (gcsKey, sourceProvider); defer the identity-key swap to a dedicated, backfilled migration (see 1.13).

1.3 dropboxLink, compressedUri & source media handling · Risk: Medium

Today	`dropboxLink` is the source of truth for the original bytes; `compressedUri` holds the GCS URI of the compressed copy the AI reads; media type is guessed from the URL extension.
After GCS	`gcsKey` holds the original in GCS; `compressedUri` stays (already GCS); a stored content type replaces extension-guessing; a generic “source URI” hides whether it's Dropbox or GCS.
Could break	Pipelines that branch on “is this a Dropbox link”; media-type detection for files without a clear extension.
To update	A single “resolve source bytes” helper in both apps, the media-type detector, and the download step used by compliance/compression.

Recommendation: Introduce one source-resolver that returns a signed GCS URL or a Dropbox link, so callers stop caring which backend it is.

1.4 Compliance check pipeline · Risk: High

Today	The web app sends the Dropbox link to the Python API; Python downloads from Dropbox, compresses, uploads to GCS, and Gemini analyzes the GCS copy.
After GCS	The web app sends a GCS reference; Python reads directly from GCS and skips the Dropbox download entirely (often faster).
Could break	The compliance-queue contract and the Celery task signature both currently require a Dropbox link.
To update	The compliance-queue payload, the Python queue handler and task, and the downloader — all to accept a GCS source alongside Dropbox.

Recommendation: Support both sources during dual-run: add the GCS branch, keep the Dropbox branch until every video is migrated.

1.5 Compliance reset & auto-queue behavior · Risk: High

Today	Changing a video's Dropbox link resets compliance + thumbnail and re-queues a check; the auto-trigger hard-fails (marks an error) if a video has no Dropbox link.
After GCS	The same reset/re-queue must fire when the GCS file is replaced, and the auto-trigger must treat a GCS-backed video as valid instead of erroring on a missing Dropbox link.
Could break	New GCS videos silently flagged “error — dropbox link missing”; reset not firing on file replace.
To update	The compliance auto-trigger in the video audit extension, and the reset-on-change detection (key off the file identity, not the Dropbox URL).

Recommendation: Update the trigger to accept `gcsKey` as a valid source early (Phase 2), with explicit tests for the replace path.

1.6 Existing “Fetch Dropbox Folder” functionality · Risk: Low

Today	A route scans the concept's Dropbox folder, creates per-file shared links, matches files to records by base name, and auto-creates missing video rows.
After GCS	Obsolete for new videos — direct upload already knows the concept and creates the record. Can be repurposed as a one-time importer only.
Could break	UI buttons and the onboarding tour that reference “Fetch Dropbox Folder”; any cron that calls it.
To update	The videos page UI, the route (retire or rename to “Import”), and the tour copy.

Recommendation: Rename to “Upload / Import Media”; keep the folder scan available only for legacy import.

1.7 Missing-video auto-create flow · Risk: Medium

Today	The folder sync auto-creates Video rows for orphan Dropbox files not yet in the database.
After GCS	Replaced by the upload flow's fill-empty-slot matching (already built): a same-named upload fills an existing empty placeholder instead of duplicating.
Could break	Duplicate video rows if both the old auto-create and the new upload-matching run on the same asset.
To update	Ensure only one creation path is active per video; disable Dropbox auto-create once upload is the default.

Recommendation: Turn off Dropbox auto-create when upload becomes the default; rely on the upload matching already in place.

1.8 Video preview & thumbnail handling · Risk: Medium

Today	Thumbnails come from the Dropbox thumbnail API (with an OpenCV frame-grab fallback); previews rely on Dropbox. Thumbnail jobs select work by filtering on the Dropbox link.
After GCS	Generate thumbnails once at upload time from the GCS object (promote the existing OpenCV path to primary); previews via signed GCS URLs.
Could break	Thumbnail jobs whose selection query filters on the Dropbox link; the thumbnail proxy route.
To update	The thumbnail tasks (source + selection query), the thumbnail proxy, and an upload-time thumbnail step.

Recommendation: Generate the thumbnail (and an optional hover-preview sprite) at upload; keep Dropbox thumbnails for legacy videos.

1.9 Sharing & permission flow · Risk: Medium

Today	Access is via Dropbox shared links, governed by Dropbox's own permissions.
After GCS	Access is governed by the app (login + roles) using short-lived signed URLs; nothing is publicly exposed by default.
Could break	Any assumption that a video URL is permanently public or freely shareable.
To update	Playback to use signed URLs (already done), plus a small share service for links that need wider access.

Recommendation: Private-by-default with signed URLs; build tokenized share pages for the few genuine external cases (see 1.11).

1.10 Existing user workflow impact · Risk: Medium

Today	Editors' habit is export → Dropbox desktop app → paste link → run fetch; reviewers use Dropbox / Replay.
After GCS	Editors drag-drop directly into Ad Genius with no context switch; reviewers use the in-app player and comments.
Could break	Muscle memory and any SOPs/onboarding that reference Dropbox; a training gap at switch-over.
To update	SOPs and onboarding docs, the videos UI, and the product tour.

Recommendation: Brief the team, keep both flows briefly, and provide a short how-to before switching the default.

1.11 External sharing / reviewer access · Risk: Medium

Today	To share with outside reviewers, editors send Dropbox links (and sometimes Replay for frame-accurate comments).
After GCS	Tokenized share pages (optional password/expiry) served via signed URLs; reviewer feedback becomes the native timestamped-comment feature.
Could break	External-reviewer workflows the day Dropbox is switched off, if the native share/comment features are not ready yet.
To update	A share-link model + public review page, and the comments feature (Phase 4).

Recommendation: Ship native share + comments before removing Dropbox — sequencing matters more than code here.

1.12 Folder structure & asset organization · Risk: Low

Today	Dropbox folders per concept; people browse assets inside Dropbox.
After GCS	GCS key prefixes per concept (already built) plus an in-app folder browser (already built); organization lives in the app.
Could break	Little — the in-app Video Library already mirrors the per-concept folder model.
To update	Nothing major; keep the prefix convention aligned with concept IDs.

Recommendation: Keep the concept-prefix convention now; add richer categories/library later (Phase 5).

1.13 Performance / statistics mapping · Risk: High

Today	Ad-performance mapping and the suggestion/learnings jobs attach videos to BigQuery rows using the normalized Dropbox link as the key.
After GCS	The mapping must key on the Dropbox-free identifier, and both old and new videos must map correctly.
Could break	Performance/attribution for new GCS videos (no Dropbox URL to join on), and winning-hooks / learnings grouping.
To update	The BigQuery concept-mapping, the suggestion-job matching, and the winning-hooks ETL upsert keys.

Recommendation: This is the heart of the identity migration — dual-write the new key, backfill, and verify parity before switching any joins.

1.14 Backward compatibility for old Dropbox videos · Risk: Medium

Today	Every existing video is Dropbox-backed.
After GCS	New videos are GCS-backed; old ones keep their Dropbox links and pipelines until (optionally) migrated.
Could break	Code that assumes exactly one source, or gaps in mixed-mode handling.
To update	Every source resolver and pipeline to handle “has gcsKey → GCS, otherwise → Dropbox”.

Recommendation: Adopt the “no gcsKey means legacy Dropbox” convention everywhere (already used in the current upload feature).

1.15 Risks if Dropbox is removed too early · Risk: High

Today	Dropbox is load-bearing for both storage and identity across the web app and the Python backend.
After GCS	It is only safe to remove once all pipelines read from GCS, the identity migration is complete, and legacy files are copied or retired.
Could break	Compliance, ad launch, suggestions, learnings, and performance — broadly — if credentials are revoked prematurely.

To update	Sequence decommissioning last; keep Dropbox read-only until a clean verification window passes.

Recommendation: Revoke Dropbox only after logs show zero Dropbox calls for a full verification window.

1.16 Recommended safe migration approach

- **Additive schema first** — add `gcsKey` (done) and `sourceProvider`/`sourceUri`; do not remove or repurpose `dropboxLink` yet.
- **Dual-run storage** — new uploads go to GCS; legacy Dropbox videos keep working untouched.
- **Dual-source pipelines** — compliance, thumbnails, launch and tagging accept both a GCS reference and a Dropbox link during the transition.
- **Identity migration with backfill** — introduce the Dropbox-free key, dual-write, backfill history, verify parity, then switch joins/constraints.
- **Features before switch-off** — ship comments, versions and share links so no reviewer workflow is lost.
- **Decommission last** — set Dropbox read-only, confirm zero usage, then revoke credentials and (optionally) bulk-copy historical files.

Part 2 — Developer effort estimate

Each phase lists scope, the files/modules likely affected, backend/frontend/database changes, testing needed, a T-shirt effort size, a risk level, and whether it is required for the MVP or can come later. Effort sizes reflect build complexity and scope.

Phase 1 — GCS upload foundation

Scope	Direct in-app video upload; store originals in GCS; create the video record immediately; add source provider/URI fields; keep Dropbox as a legacy fallback.
Files / modules	The creative-video storage library; the upload-url / finalize / play-url / replace-url routes; the Video Library UI; the videos API; the Prisma schema.
Backend	Signed upload/read/delete (done); add `sourceProvider` + `sourceUri`; finalize marks the video ready.
Frontend	Video Library folder grid + upload modal (done).
Database	`gcsKey` (done); add `sourceProvider`, `sourceUri`; make `dropboxLink` optional.
Testing	Large-file upload, replace, delete-removes-GCS-object, and a mixed legacy/GCS listing.
Effort	**Small–Medium (mostly already built)**
Risk	**Low**
MVP?	**Required — this is the foundation**

Phase 2 — Compliance pipeline migration

Scope	Compliance queue supports both Dropbox and GCS; update the Python payload handling; replace the Dropbox freshness check with GCS metadata/hash-based freshness; keep compliance reset correct.
Files / modules	The web compliance-queue and video audit extension; the Python compliance service, queue handler and Celery task; the media-type detector; the shared downloader.
Backend	Accept a GCS source in the payload and task; read bytes from GCS; compute freshness from the GCS object generation/hash; keep the Dropbox branch.

Frontend	Minimal — ensure new GCS videos display the correct compliance state.
Database	Store content type and a content hash on the video; optionally a compression state.
Testing	Compliance pass/fail for a GCS video; reset on replace; dual-source handling; result parity with the Dropbox path.
Effort	**Large**
Risk	**High (correctness + external Python/Gemini pipeline)**
MVP?	**Required for full cutover — can run dual-source**

Phase 3 — UI & workflow changes

Scope	Replace or rename “Fetch Dropbox Folder”; add the Upload Media flow; add preview/thumbnail handling; support old Dropbox videos and new GCS videos in the same UI.
Files / modules	The videos page; the fetch-concept-folder route; the Video Library UI; the thumbnail route and Python thumbnail tasks; the tour.
Backend	Retire/rename the sync route; add upload-time thumbnail generation.
Frontend	A unified list of Dropbox + GCS videos; the upload-media UI (done); preview.
Database	Reuse `thumbnailUrl`; upload state in the video meta (done).
Testing	Mixed-list rendering; thumbnails for both source types; tour copy.
Effort	**Medium**
Risk	**Low–Medium**
MVP?	**Partly done — rename + thumbnails required before switch-off**

Phase 4 — Dropbox-like collaboration features

Scope	Timestamped comments; version history; shareable links with permissions; external reviewer access; download/view permissions.
Files / modules	New comment / version / share models and APIs; the video player UI; public share/review pages.
Backend	Comment CRUD; version tracking; share tokens with optional password/expiry; signed URLs.
Frontend	Player timeline markers + comment threads; version list/restore; a share dialog; a public review page.
Database	New tables: video comments, video versions, video shares.
Testing	Comment anchoring/seeking; version restore; share expiry/password; permission checks.
Effort	**Large**
Risk	**Medium**
MVP?	**Later — needed before removing Dropbox/Replay for external reviewers**

Phase 5 — Asset library & future DAM features

Scope	Folder/category structure; a centralized asset library; smart tagging / AI search; performance mapping; a top-performer dashboard.

Files / modules	A library UI; the tagging pipeline (Python + Gemini, partly exists); Pinecone-based search; performance mapping; a dashboard.
Backend	AI tagging + embeddings; search endpoints; performance mapping on the new identity.
Frontend	Library browse/search UI; the dashboard.
Database	Tags (exist), embedding/vector references, performance links.
Testing	Search relevance; tagging accuracy; dashboard correctness.
Effort	**Very Large**
Risk	**Medium**
MVP?	**Later — future value, not required to leave Dropbox**

Part 3 — Team requirements & how we'd rebuild each

The team's workflow does not depend on Dropbox alone — it also relies on **Dropbox Replay** (video review and comments) and **Air** (asset management and AI search). Each requirement below shows what the tool provides today, the flow we would have to build, what Ad Genius must build itself, and why it is heavy effort. (This is the same work as Phases 4–5, seen through the requirement lens.)

> The key point behind this whole section: **Dropbox and Air are not just storage.** They bundle storage **plus** an application layer — a video player, a comment box, playback-timestamp capture, a comments-and-permissions database, and (for Air) an AI indexing layer. **GCS gives us storage only.** Every one of those application-layer pieces has to be built inside Ad Genius. That collaboration and intelligence layer — not the file storage — is where the real effort lives.

R1 — Direct in-app upload (the trigger for everything else)

What the team wants	Upload videos directly inside Ad Genius instead of uploading to Dropbox and pasting a link.
How it works today	Editors upload to the Dropbox desktop app / web, copy the shared link into Ad Genius, then run “Fetch Dropbox Folder” to match files to records.
The flow we'd build	User uploads video in Ad Genius ↓ Video is stored in GCS (private bucket, per-concept prefix) ↓ Ad Genius creates a Video record in the DB immediately ↓ Upload is confirmed and the video appears under its concept
What Ad Genius must build	The in-app uploader, the signed direct-to-GCS upload, and the record/folder mapping. Most of this is already built.
Why it's heavy effort	Small on its own — but it is the domino that forces everything else. Once files leave Dropbox, every pipeline, the identity scheme, thumbnails, ad launch and reporting all have to change too.
Effort	**Small to build · Large ripple effect**

R2 — Timestamped comments (replaces Dropbox Replay)

What the team wants	Feedback pinned to the exact second/frame — e.g. “at 00:12, please change this frame.”

How it works today	Dropbox is not getting the comment from the raw file — it provides a collaboration layer on top: its own video player, the comment box, the current-playback timestamp, the comments database, and reviewer permissions. Timestamp comments are an application feature, not a storage feature.
The flow we'd build	User uploads video in Ad Genius ↓ Video is stored in GCS ↓ Ad Genius creates a Video record in the DB ↓ Ad Genius generates a secure, time-limited signed URL ↓ Frontend plays it in an HTML5 <video> player ↓ Reviewer pauses at a timestamp and adds a comment ↓ Ad Genius saves the comment + timestamp in the DB ↓ Clicking a comment seeks the video back to that timestamp
What Ad Genius must build	GCS only stores the file. We must build the entire collaboration layer ourselves: the player, the comment box and threads, capturing the playback time, the comments database, timeline markers, resolve state, and reviewer permissions. (Signed URLs give time-limited access to a private object, so the bucket never needs to be public.)
Why it's heavy effort	Dropbox Replay has no public API, so none of this can be reused — it is a from-scratch build, and it must feel as good as Replay or editors will not adopt it.
Effort	**Large**

R3 — Version history

What the team wants	Upload a new version of a video without losing previous ones or their comments.
How it works today	Dropbox keeps versions automatically as a built-in storage + app feature, and lets you view or restore an older one.
The flow we'd build	Editor uploads a new version ↓ Stored as a new object in GCS (versioned key) ↓ Ad Genius adds a Version row linked to the same Video ↓ Old versions and their comments are kept ↓ Restore = switch the “current” pointer to a chosen version
What Ad Genius must build	The version data model, a new object per version, UI to list / compare / restore, and logic so comments and compliance follow the video across versions. GCS object versioning can act as a backstop.
Why it's heavy effort	It is an application feature we now own end-to-end, and it interacts directly with the identity migration (each version must map to the same video).
Effort	**Large**

R4 — Shareable links with permissions

What the team wants	Share a video with specific people and control what they can do with it.

How it works today	Dropbox generates shareable links with permission controls (public / specific people / password / expiry) out of the box.
The flow we'd build	User creates a share for a video ↓ Ad Genius issues a share token + a signed-URL policy ↓ Recipient opens a tokenized share page ↓ Optional password / expiry / audience is checked ↓ Video is served via a short-lived signed URL
What Ad Genius must build	A share data model, the tokenized public page, the permission checks, signed-URL issuance, and revocation — the permission layer Dropbox gave us for free.
Why it's heavy effort	It is security-sensitive and adds a new public surface we have to protect and audit.
Effort	**Medium to Large**

R5 — External reviewer access & download/view permissions

What the team wants	Let outside reviewers view (and sometimes download) a video without an Ad Genius account.
How it works today	Editors send a Dropbox link; Dropbox handles the external, no-login access and its view/download permissions.
The flow we'd build	Editor creates a review link (view-only or download) ↓ External reviewer opens the public review page ↓ Ad Genius serves the video via a short-lived signed URL ↓ Optional watermark; every access is logged
What Ad Genius must build	Per-link view-only vs download flags, an external (no-login) review page, download gating, and an access audit trail.
Why it's heavy effort	A public, un-authenticated surface brings abuse and security considerations, plus external-access auditing we do not have today.
Effort	**Medium**

R6 — Hover preview (replaces an Air feature)

What the team wants	Preview and scrub a video on hover without opening it.
How it works today	Air auto-generates hover/scrub previews as part of its DAM — again, an application feature on top of storage.
The flow we'd build	Video is uploaded to GCS ↓ Ad Genius generates a thumbnail sprite / short preview clip (ffmpeg/OpenCV) ↓ Preview stored in GCS ↓ Library scrubs the sprite on hover
What Ad Genius must build	A preview-generation step in the media pipeline, storage for the previews, and a fast hover UI in the library.

Why it's heavy effort	A new per-upload processing step with storage and compute cost, and it has to be fast to feel like Air.
Effort	**Medium**

R7 — Smart tagging / AI search (replaces an Air feature)

What the team wants	Find specific moments or topics inside long videos, regardless of the filename.
How it works today	Air runs its own AI indexing and search over the assets — and this is not exposed through its public API.
The flow we'd build	Video uploaded to GCS ↓ Gemini analyzes it → tags + a timestamped transcript ↓ Embeddings stored in a vector database (Pinecone, already in the stack) ↓ Search UI queries the vectors ↓ Results deep-link to the exact moment in the video
What Ad Genius must build	The entire AI pipeline — analysis, embeddings, indexing and re-indexing — plus the search API and UI.
Why it's heavy effort	Air's AI search is not in its API, so the whole pipeline is ours to build and tune: accuracy, cost, re-indexing, and keeping it current as the library grows.
Effort	**Very Large**

R8 — Centralized asset library (replaces an Air feature)

What the team wants	One place to browse and reuse UGC, B-roll and AI-generated assets across concepts.
How it works today	Air is the centralized DAM the team browses and reuses assets from.
The flow we'd build	All GCS assets are indexed ↓ Library UI browses & searches them (uses R7) with previews (uses R6) ↓ An asset can be reused by linking it to another concept
What Ad Genius must build	A library UI, a cross-concept reuse model, and organization (categories/collections) — combining R6 + R7 + browsing into one product surface.
Why it's heavy effort	This is effectively building a mini-DAM: an ongoing product area, not a one-off feature.
Effort	**Very Large**

What this migration really involves

Put together, this is not a storage swap — it is a **large, complexity-adding programme.** It spans **two codebases** (the Next.js app and the Python backend), a **high-risk data migration** (re-pointing the video identity that suggestions, learnings and reporting depend on), and **rebuilding features from three separate products** — Dropbox, Dropbox Replay and Air — two of which offer no API. The points below capture what it costs us during the transition and how to sequence it so nothing breaks.

What we lose (until rebuilt)

- Dropbox Replay's ready-made frame-accurate comments and Dropbox's built-in version history.
- The desktop-sync convenience editors are used to, and Air's polished AI search / hover previews.
- Some team familiarity — a short adjustment period is unavoidable.

What should NOT be removed immediately

- The `dropboxLink` field and the Dropbox-URL identity — keep until the new identifier is backfilled and verified.
- The Dropbox download pipelines (compliance, thumbnails, launch, tagging) — keep until every video reads from GCS.
- The folder-sync route — keep (read-only) until legacy assets are imported or retired.
- Dropbox credentials — revoke last, only after a clean zero-usage verification window.

Safest rollout strategy

A phased, dual-run rollout (Phases 1 → 5): additive schema first, dual-source pipelines during transition, an identity migration with backfill and parity checks, native collaboration features before switch-off, and decommissioning Dropbox last. Every step is reversible until the final credential revocation, so the team is never blocked and there is no “big bang” moment.